

Tema 8 — Operaciones de entrada-salida

Contenido teórico

Objetivos

- **Entender la entrada-salida** como intercambio de datos entre el programa, el usuario y el sistema de ficheros.
- **Abrir, leer, escribir y cerrar** ficheros de texto usando **open()** y **with**.
- **Elegir modos de apertura adecuados:** lectura, escritura, añadido, creación exclusiva y binario.
- **Trabajar con rutas**, codificación, CSV, ficheros binarios y serialización básica.
- **Aplicar gestión de errores** para evitar fallos poco claros en operaciones de disco.

Mapa del Tema 8

- **Primero revisaremos entrada/salida estándar:** **print()**, **input()** y datos procedentes del sistema.
- **Después veremos rutas**, **open()**, modos de apertura y gestión automática del cierre con **with**.
- **A continuación trabajaremos lectura y escritura** de texto, CSV y ficheros binarios.
- **Terminaremos con buenas prácticas y errores frecuentes** en scripts de TI.

```
entrada usuario → validación
                    ↓
                operación Python
                    ↓
    fichero texto / CSV / binario
                    ↓
                lectura, copia o reporte
```

Qué entendemos por entrada-salida

- **Entrada es cualquier dato que llega al programa:** teclado, fichero, argumento, API o resultado de otro proceso.
- **Salida es la información que el programa produce:** pantalla, fichero, informe, log o dato exportado.
- En este tema nos centramos en operaciones locales sencillas con ficheros y formatos habituales.
- La gestión de errores del tema anterior se aplica directamente a rutas, permisos y formatos.

```
entrada = "443"
puerto = int(entrada)

reporte = f"nginx:{puerto}"
print(reporte)
```

Entrada y salida estándar

- **print()** es la salida más simple: muestra información en consola o en la salida del notebook.
- **input()** solicita texto al usuario y siempre devuelve str.
- **NOTA:** En notebooks conviene limitar input() para evitar bloqueos o dependencias interactivas innecesarias. Para las prácticas con ficheros usaremos valores definidos en el código o datos leídos desde archivos para evitar el bloqueo en los notebooks.

```
usuario = "admin"
servicio = "ssh"

print("Usuario", usuario, sep="=")
print("Comprobando", end="...")
print(servicio)
```

Rutas y directorio de trabajo

Una ruta identifica dónde está un fichero o directorio.

- **Ejemplos:**
 - **C:\Users\pedrolopezs\Documents\texto.txt** (Windows)
 - **/home/user/Documents/texto.txt** (Linux)

Las rutas relativas dependen del directorio desde el que se ejecuta el script.

- **Ejemplos:**
 - **pwd** (devuelve en Windows C:\Users\pedrolopezs)
 - **Documents\texto.txt** (Ruta relativa al ejemplo anterior)
 - **pwd** (devuelve en linux /home/user)
 - **Documents/texto.txt** (Ruta relativa al ejemplo anterior)

Tradicionalmente la gestión de rutas se ha hecho mediante operaciones de split y concatenación de cadenas.

A partir de **Python3.4** aparece la librería **pathlib** con la clase **Path**

- Detecta las diferencias de Sistema Operativo y puede trabajar con rutas en Windows, Linux o OSX.
- Permite construir rutas de forma más clara y portable que concatenar cadenas.
- Antes de leer o escribir en ficheros conviene tener claro dónde se crearán éstos.

```
from pathlib import Path

# la clase Path crea una
# instancia de tipo
# "ruta relativa"
carpeta = Path("Tema08/data")

# Podemos concatenar dicha
# ruta a un nombre de fichero
ruta = carpeta / "file.txt"

# Mostramos la ruta relativa
print(ruta)

# El método resolve() calcula
# la ruta absoluta a partir
# al pwd actual
print(ruta.resolve())
```

Operaciones sobre rutas, directorios y ficheros

La librería `pathlib` y su clase `Path` pueden hacer otras operaciones previas a la lectura/escritura sobre ficheros:

- **`Path.exists()`** : comprueba si una ruta existe.
- **`Path.is_file()` y `Path.is_dir()`** : validan el tipo de recurso.
- **`Path.mkdir(exist_ok=True)`** : Crea el directorio si no existe. (no da error si existe)
- **`Path.read_text(codificación)`** : Lee el contenido de un fichero sin usar `open()/close()` o `with` (Lo veremos a continuación)
- **`Path.write_text(texto, codificación)`**: escribe en un fichero sin usar `with`
- **`Path.with_name("nombre")`** : Cambia en una ruta la parte del nombre del fichero
- **`Path.rename()`** : permite cambiar nombre o mover un fichero dentro del mismo sistema.
- **`Path.unlink()`** : elimina un fichero.
- **`Path.suffix`** : atributo que muestra la extensión del fichero.

```
from pathlib import Path

carpeta = Path("Tema08/data")
# Si existe el directorio no falla
# Si no existe lo crea
carpeta.mkdir(exist_ok=True)
ruta = carpeta / "file1.txt"

# Comprobaciones
print("Ruta: ", ruta)
print("Carpeta existe: ", carpeta.exists())
print("Carpeta es dir: ", carpeta.is_dir())
print("Fichero existe: ", ruta.exists())

# Escribe en la ruta (lo veremos a continuación)
print("Escribimos en Fichero")
ruta.write_text("pruebal", encoding="utf-8")
print("Fichero existe: ", ruta.exists())
print("Fichero es file: ", ruta.is_file())
print("Fichero extensión: ", ruta.suffix)
print("Contenido fichero: ", ruta.read_text())

# Cambiamos el nombre de la ruta y renombramos
nueva = ruta.with_name("file1_renamed.txt")
print("Ruta: ", nueva)
ruta.rename(nueva)

if nueva.exists():
    nueva.unlink()
print("Fichero existe: ", nueva.exists())
```

`open()` / `close()` : abrir y cerrar un fichero

`open()` : devuelve un objeto fichero que permite leer o escribir datos.

- Sus argumentos principales son:
 - La ruta
 - El modo de apertura
 - La codificación en texto.

close() : Cierra el fichero

Modos de apertura:

- **r** : lectura. Falla si el fichero no existe.
- **w** : escritura. Crea o sobrescribe desde cero.
- **a** : append. Escribe al final del contenido.
- **x** : creación exclusiva. Falla si ya existe.
- **b** : modo binario. / **t** : modo texto (default).
- **+** : lectura y escritura.

NOTAS:

- Si no se cierra el fichero, los datos pueden quedar sin volcar o el recurso bloqueado. Se recomienda usar **with**.
- La codificación indica cómo se representan los caracteres en bytes: Para texto actual conviene usar **utf-8** de forma explícita. Sólo habría que usar otras codificaciones en ficheros más antiguos.

```
from pathlib import Path

carpeta = Path("Tema08/data")
carpeta.mkdir(exist_ok=True)
ruta = carpeta / "file2.txt"

# Abrimos el fichero de escritura
fichero = open(ruta, "w", encoding="utf-8")
fichero.write(f"Línea 1\n")
fichero.write(f"Línea 2\n")
fichero.write(f"Línea 3\n")
fichero.close()

# Abrimos el fichero de lectura
fichero = open(ruta, "r", encoding="utf-8")
todo_el_texto = fichero.read()
print(todo_el_texto.strip())
fichero.close()
```

Gestión automática de ficheros con with

with crea un contexto de trabajo para el fichero.

- **Sintaxis tradicional:**
 - fichero = open(ruta, modo, encoding="utf-8")
 - fichero.close()
- **Sintaxis con with:**
 - with open(ruta, modo, encoding="utf-8") as fichero:

Lo más interesante es que al salir del bloque, Python cierra el fichero aunque ocurra una excepción.

- Reduce errores y evita olvidarse de close().
- Es la forma recomendada para leer y escribir ficheros.

```
from pathlib import Path

carpeta = Path("Tema08/data")
carpeta.mkdir(exist_ok=True)
ruta = carpeta / "file3.txt"
```

```
# Abrimos el fichero de escritura con with
with open(ruta, "w", encoding="utf-8") as fichero:
    for i in range(1, 4):
        fichero.write(f"Línea {i}\n")

# Abrimos el fichero de lectura con with
with open(ruta, "r", encoding="utf-8") as fichero:
    for línea in fichero:
        print(línea.strip())
```

Escribir en ficheros de texto

Escritura en ficheros:

- En modo **w** se crea el fichero si no existe y se reemplaza si ya existía.
- **write()** escribe una cadena exactamente como se le pasa.
- Si queremos saltos de línea, hay que incluir **\n**
- **encoding="utf-8"** evita problemas con acentos y caracteres especiales.

Añadir datos a un fichero:

- El modo **a** abre el fichero y coloca la escritura al final.
- Es útil para registrar eventos simples o añadir líneas a un informe.
- No borra lo que ya existía.
- Para logs reales se suele usar el módulo logging, que veremos más adelante.

```
from pathlib import Path

carpeta = Path("Tema08/data")
carpeta.mkdir(exist_ok=True)
ruta = carpeta / "servicios1.txt"

# Apertura en modo w (Crea el fichero si no existe)
with open(ruta, "w", encoding="utf-8") as fichero:
    fichero.write(f"{'Servicio':<8} {'Puerto':>6} {'Estado':>6}\n")
    fichero.write(f"{'ssh':<8} {'22':>6} {'OK':>6}\n")
    fichero.write(f"{'https':<8} {'443':>6} {'OK':>6}\n")
    fichero.write(f"{'http':<8} {'80':>6} {'NA':>6}\n")

# Apertura en modo a (Agrega líneas al final del fichero)
with open(ruta, "a", encoding="utf-8") as fichero:
    fichero.write(f"{'ntp':<8} {'123':>6} {'OK':>6}\n")
    fichero.write(f"{'dns':<8} {'53':>6} {'OK':>6}\n")

# Lectura
with open(ruta, "r", encoding="utf-8") as fichero:
    todo_el_texto = fichero.read()
    print(todo_el_texto.strip())
```

Leer en ficheros de texto

En modo **r** podemos leer el contenido de un fichero con diferentes métodos:

- **read()** lee todo el contenido como una única cadena.
- **readline()** lee una línea cada vez y avanza el puntero.
- **readlines()** devuelve una lista con todas las líneas.

NOTAS:

- Para ficheros grandes suele ser mejor recorrer el fichero línea a línea.
- Los **print()** siempre imprimen con salto de línea, por lo que eliminamos los saltos del fichero con **strip()** para que no sean redundantes.

```
from pathlib import Path

carpeta = Path("Tema08/data")
carpeta.mkdir(exist_ok=True)
ruta = carpeta / "servicios1.txt"

print("Leemos todo el contenido de una vez con read()")
with open(ruta, "r", encoding="utf-8") as fichero:
    todo_el_contenido = fichero.read()
print(todo_el_contenido.strip())

print("Leemos la cabecera y primera línea con readline()")
with open(ruta, "r", encoding="utf-8") as fichero:
    cabecera = fichero.readline()
    primer_servicio = fichero.readline()
print("Cabecera          : ", cabecera.strip())
print("Línea en la posición 1: ", primer_servicio.strip())

print("Leemos el fichero en una lista con readlines()")
with open(ruta, "r", encoding="utf-8") as fichero:
    lista_de_líneas = fichero.readlines()
print("Línea en la posición 2: ", lista_de_líneas[2].strip())
print("Línea en la posición 3: ", lista_de_líneas[3].strip())
print("Número total de líneas:", len(lista_de_líneas))
```

Lectura de ficheros línea a línea

Los tres métodos **readxx()** anteriores cargan el fichero entero en memoria.

- Esto es computacionalmente poco eficiente, sobre todo en ficheros grandes.
- Leer línea a línea integrando la lectura en un bucle permite procesar ficheros sin cargarlos enteros.
- **strip()** elimina el salto de línea y espacios sobrantes en los extremos.
- **split()** puede separar campos cuando el formato es sencillo.

Este patrón es habitual para logs, inventarios o ficheros de configuración simples.

La asignación **_ = fichero.readline()** se usa para descartar la variable que no nos interesa, de lo contrario, en modo interactivo saldría por pantalla.

```
from pathlib import Path

carpeta = Path("Tema08/data")
carpeta.mkdir(exist_ok=True)
ruta = carpeta / "servicios1.txt"

print("Bucle iterando línea a línea y usando strip()")
with open(ruta, "r", encoding="utf-8") as fichero:
    for línea in fichero:
        print(línea.strip())

print("Bucle iterando línea a línea y usando split()")
with open(ruta, "r", encoding="utf-8") as fichero:
    _ = fichero.readline() # Saltamos la cabecera
    for línea in fichero:
        campos = línea.split() # Trocea la línea
        print(f"{campos[0]} ({campos[1]}) -> Estado: {campos[2]}")
```

Otras operaciones

Copiar un fichero de texto

- Copiar texto consiste en leer de un fichero y escribir en otro.
- El fichero de destino debe abrirse en modo escritura.
- Procesar por líneas evita cargar todo el contenido en memoria.
- En operaciones reales de copia de archivos completos existe **shutil.copyfile()**.

Reposicionar el cursor con seek()

- Cuando se lee un fichero, el cursor avanza por el contenido.
- Si queremos volver al principio del mismo objeto fichero, usamos **seek(0)**.
- En código real suele ser más simple abrir de nuevo el fichero si hace falta.

```
from pathlib import Path

carpeta = Path("Tema08/data")
carpeta.mkdir(exist_ok=True)
origen = carpeta / "servicios1.txt"
destino = carpeta / "servicios2.txt"

# Copiar un fichero de entrada en un fichero de salida
with open(origen, encoding="utf-8") as entrada, \
    open(destino, "w", encoding="utf-8") as salida:
    for línea in entrada:
        salida.write(línea)

# Reposicionar el puntero de un fichero al inicio
with open(origen, "r", encoding="utf-8") as fichero:
    print("Primera impresion")
    fichero_entero1 = fichero.read()
    print(fichero_entero1.strip())
    _ = fichero.seek(0) # posiciona e ignora la salida
    print("Segunda impresion")
    fichero_entero2 = fichero.read()
    print(fichero_entero2.strip())
```

CSV: datos tabulares

Los ficheros **CSV** almacenan datos en filas y columnas usando un separador.

- Es habitual para exportar inventarios, reportes o datos de hojas de cálculo.
- Python incluye el módulo **csv** para leer y escribir estos ficheros con seguridad.
- La clase **csv.writer(fichero, delimitador)** crea un objeto escritor con el que trabajamos para escribir las líneas del fichero
- El método **writerow()** crea líneas en el writer
- El parámetro **newline=""** es obligatorio ponerlo en ficheros csv al abrir en modo escritura **"w"**. Si se omite, la librería **csv** insertará dobles saltos de línea (**\r\r\n**) en SO como Windows, rompiendo el formato al abrirlo en Excel.

En España y entornos europeos se puede encontrar punto y coma como separador en lugar de coma simple. Esto es definible.

```
import csv
from pathlib import Path
```

```

carpeta = Path("Tema08/data")
carpeta.mkdir(exist_ok=True)
ruta = carpeta / "servicios1.csv"

with open(ruta, "w", newline="", encoding="utf-8") \
    as filecsv:
    escritor = csv.writer(filecsv, delimiter=",")
    escritor.writerow(["servicio", "puerto", "estado"])
    escritor.writerow(["ssh", 22, "OK"])
    escritor.writerow(["http", 80, "OK"])

```

Escribir CSV con DictWriter

La clase **DictWriter** permite escribir cada fila desde un diccionario, lo que evita tener que trabajar con posiciones e índices.

- Las cabeceras definen el orden y los nombres de columnas.
- Es más legible que trabajar con posiciones cuando los campos tienen nombre.
- Encaja bien con datos estructurados de inventario.

```

import csv
from pathlib import Path

carpeta = Path("Tema08/data")
carpeta.mkdir(exist_ok=True)
ruta = carpeta / "servicios2.csv"

campos = ["servicio", "puerto", "estado"]
fila1 = {"servicio": "ssh", "puerto": 22, "estado": "OK"}
fila2 = {"servicio": "http", "puerto": 80, "estado": "OK"}

with open(ruta, "w", newline="", encoding="utf-8") as filecsv:
    escritor = csv.DictWriter(filecsv, fieldnames=campos, delimiter=",")
    escritor.writeheader()
    escritor.writerow(fila1)
    escritor.writerow(fila2)

```

Leer CSV con reader y DictReader

- El objeto **csv.reader** devuelve cada fila como una lista.
- El objeto **csv.DictReader** usa la primera fila como cabecera y devuelve diccionarios.
- **DictReader** suele ser más claro cuando las columnas tienen nombre.
- Los valores leídos desde **CSV** llegan como texto y deben convertirse si hace falta.

```

import csv
from pathlib import Path

carpeta = Path("Tema08/data")
carpeta.mkdir(exist_ok=True)
ruta = carpeta / "servicios1.csv"

print("Ejemplo con reader")
with open(ruta, "r", encoding="utf-8") as f:
    lector = csv.reader(f, delimiter=",")
    cabecera = next(lector) # Ignoramos la cabecera

```



```

for fila in lector:      # Cada fila es una lista
    servicio, puerto, estado = fila
    print(f"Servicio: {servicio} -> {puerto}: {estado}")

print("Ejemplo con DictReader")
with open(ruta, "r", newline="", encoding="utf-8") as filecsv:
    lector = csv.DictReader(filecsv, delimiter=",")
    for fila in lector:      # Cada fila es un diccionario
        puerto = int(fila["puerto"])
        print(
            f"Servicio: {fila['servicio']} -> "
            f"{fila['puerto']}: {fila['estado']}"
        )

```

Transformar datos al leer

La lectura de ficheros suele combinarse con limpieza, conversión y validación.

- **strip()**, **lower()** y otros métodos normalizan la cadena.
- **int()** o **float()** convierten campos numéricos.
- **try/except** permite saltar o registrar líneas con formato incorrecto.

En el ejemplo vemos un proceso de limpieza, normalización y transformación de datos de un csv

```

import csv
from pathlib import Path

carpeta = Path("Tema08/data")
carpeta.mkdir(exist_ok=True)
ruta = carpeta / "servicios3.csv"

try: # Apertura segura del fichero
    with open(ruta, "r", newline="", encoding="utf-8") as filecsv:
        lector = csv.DictReader(filecsv, delimiter=",")
        for fila in lector:
            # Limpieza de datos sobrantes valores con .strip()
            servicio = fila["servicio"].lower().strip()
            puerto_txt = fila["puerto"].lower().strip()
            try: # Conversión y validación del puerto con try/except
                puerto = int(puerto_txt)
            except ValueError:
                print(f"[ERROR]: '{servicio}': '{puerto_txt}'")
            else:
                print(f"Válido: -> Servicio {servicio} | Puerto: {puerto}")
except FileNotFoundError:
    print(f"[CRÍTICO] El archivo de datos no existe en la ruta: {ruta}")

```

JSON como formato de intercambio: Escritura

JSON representa datos estructurados con listas y diccionarios.

- Es muy común en APIs, herramientas cloud y ficheros de configuración.
- El módulo **json** pertenece a la biblioteca estándar y nos permite interpretar estos ficheros
- **json.dump()** toma el objeto Python y lo vuelca en el archivo.
- **indent=4** : usa cuatro caracteres de indentación (Por defecto pondría el JSON en una sola línea)

- **ensure_ascii=False** : Compatibilidad con caracteres especiales fuera del inglés. Evita forzar el estándar ASCII, combinado con la apertura en modo "utf-8" evita por ejemplo que "Dirección" se escriba como "Direcci\u00f3n"

```
import json
from pathlib import Path

carpeta = Path("Tema08/data")
carpeta.mkdir(exist_ok=True)
ruta = carpeta / "servicios1.json"

# Diccionario con los servicios
servicios = [
    {"servicio": "ssh", "puerto": 22, "estado": "OK"},
    {"servicio": "http", "puerto": 80, "estado": "OK"},
    {"servicio": "https", "puerto": 443, "estado": "ERROR"}
]

# Escritura en el JSON
try:
    with open(ruta, "w", encoding="utf-8") as filejson:
        json.dump(servicios, filejson, indent=4, ensure_ascii=False)
        print(f"Fichero guardado con éxito en: {ruta}\n")
except IOError as e:
    print(f"[ERROR] No se pudo escribir el archivo: {e}\n")
```

JSON como formato de intercambio: Lectura

En lectura usamos el mismo módulo con el método **json.load()**

- Devuelve una lista de diccionarios.
- Por lo tanto, podemos iterar sobre dicha lista de forma totalmente coherente sin tener que hacer ninguna operación adicional

```
import json
from pathlib import Path

# Configuración de rutas (manteniendo la coherencia de tu proyecto)
carpeta = Path("Tema08/data")
carpeta.mkdir(exist_ok=True)
ruta = carpeta / "servicios1.json"

# Lectura del JSON
try:
    with open(ruta, "r", encoding="utf-8") as filejson:
        servicios = json.load(filejson)
        for fila in servicios: # Cada elemento es un diccionario
            servicio = fila["servicio"]
            puerto = fila["puerto"]
            estado = fila["estado"]
            print(f"Servicio: {servicio} -> Puerto: {puerto} [{estado}]")
except FileNotFoundError:
    print(f"[CRÍTICO] El archivo no existe en la ruta: {ruta}")
except json.JSONDecodeError:
    print("[CRÍTICO] El archivo JSON está corrupto o tiene errores.")
```

Serialización binaria de objetos con pickle

pickle permite guardar objetos de Python en un formato binario propietario y recuperarlos después de forma exacta.

- **pickle.dump()** : Serializa el objeto de Python y lo vuelca tal cual en el archivo binario abierto en modo escritura ("**wb**").
- **pickle.load()** : Lee el archivo binario ("**rb**") y reconstruye el objeto original en memoria con sus tipos de datos nativos intactos.

Advertencia de seguridad: Nunca se deben cargar archivos pickle de origen no confiable. Al deserializar el archivo, este puede ejecutar código arbitrario malicioso de forma automática en el sistema.

- Para intercambio de datos entre sistemas es preferible usar formatos abiertos como **CSV** o **JSON**.

```
import pickle
from pathlib import Path

carpeta = Path("Tema08/data")
carpeta.mkdir(exist_ok=True)
ruta = carpeta / "servicios1.pkl"

#Diccionario con los servicios
servicios = [
    {"servicio": "ssh", "puerto": 22, "estado": "OK"},
    {"servicio": "http", "puerto": 80, "estado": "OK"}
]

print("Escribiendo Binario (Serialización)")
with open(ruta, "wb") as filepickle:
    pickle.dump(servicios, filepickle)

print("Leyendo Binario (Deserialización)")
with open(ruta, "rb") as filepickle:
    datos = pickle.load(filepickle)
    for fila in datos:
        print(f"Servicio: {fila['servicio']} -> {fila['puerto']}")
```

Ficheros binarios: Escritura

Un fichero binario contiene bytes, no texto plano.

- Se abre de escritura con **wb**.
- Imágenes, ejecutables, documentos o volcados de datos suelen tratarse como binarios.
- Cuando trabajamos en binario no se usa **encoding**.

En el ejemplo: Usamos una imagen JPG original para insertarle un mensaje oculto y reconocible mediante una marca de inicio al final del contenido binario del fichero

```
from pathlib import Path

imagen_original = Path("Tema08/data/gatito.jpg")
imagen_modificada = Path("Tema08/data/gatito2.jpg")

# Inserción del mensaje
# Leemos la imagen original
with open(imagen_original, "rb") as fichero:
    bytes_imagen = fichero.read()
# Definimos el mensaje con una marca inicial reconocible
mensaje = "##INICIO##\nMarca de seguridad\n"
```

```
# Codificamos el mensaje
bytes_mensaje = mensaje.encode("utf-8")
with open(imagen_modificada, "wb") as fichero:
    # Insertamos el mensaje en la imagen original
    fichero.write(bytes_imagen + bytes_mensaje)
print(";Mensaje insertado en la imagen con éxito!")
```

Ficheros binarios: Lectura

Un fichero binario contiene bytes, no texto plano.

- Se abre de lectura con **rb**.

En el ejemplo: Extraemos el mensaje del JPG modificado usando la marca reconocible como punto para separar el Contenido de la imagen del mensaje codificado.

```
from pathlib import Path

imagen_original = Path("Tema08/data/gatito.jpg")
imagen_modificada = Path("Tema08/data/gatito2.jpg")

# Extracción del mensaje
# Leemos la imagen modificada
with open(imagen_modificada, "rb") as fichero:
    contenido_binario = fichero.read()
# Codificamos el inicio reconocible del mensaje
marcador = "##INICIO##".encode("utf-8")
# Usamos el marcador para separar el JPG del Mensaje
partes = contenido_binario.split(marcador)
# Extraemos la parte final que contiene el mensaje
bytes_mensaje_puros = partes[-1]
# Decodificamos el mensaje
mensaje_extraido = bytes_mensaje_puros.decode("utf-8")

print("--- Mensaje oculto recuperado ---")
print(mensaje_extraido)
```

Cuándo usar cada formato

- **Texto plano:** mensajes, logs simples, notas o configuraciones muy sencillas.
- **CSV:** datos tabulares que pueden abrirse en hojas de cálculo o intercambiarse fácilmente.
- **JSON:** datos estructurados con listas y diccionarios.
- **Binario:** datos no textuales o serialización controlada.
- **pickle:** solo para objetos Python y fuentes confiables.

```
formato = "csv"

if formato == "txt":
    destino = "reporte.txt"
elif formato == "csv":
    destino = "reporte.csv"
else:
    destino = "reporte.json"

print(destino)
```

Buenas prácticas y Errores Frecuentes

Buenas Prácticas:

- Usar **with** para cerrar recursos automáticamente.
- Indicar **encoding="utf-8"** en ficheros de texto.
- Usar **pathlib** hace el código más claro. No construir rutas concatenando cadenas.
- No capturar todos los errores sin registrar la causa.
- No usar **pickle** con ficheros de origen desconocido.

Errores Frecuentes.

- Abrir en modo **w** pensando que se añadirá contenido: **w** sobrescribe.
- Olvidar saltos de línea al usar **write()**.
- No cerrar el fichero cuando no se usa **with**.
- Leer números desde **CSV** y olvidar que llegan como texto.
- Asumir que el script se ejecuta siempre desde el mismo directorio.

Errores en la Ejecución:

- **FileNotFoundError**: la ruta no existe o el nombre no coincide.
- **PermissionError**: el usuario no tiene permisos suficientes.
- **IsADirectoryError**: se intentó abrir un directorio como si fuera un fichero.
- **UnicodeDecodeError**: el fichero no está codificado como esperamos.

```
from pathlib import Path

ruta = Path("datos") / "salida.txt"
try:
    with ruta.open("w", encoding="utf-8") as f:
        f.write("OK\n")
except FileNotFoundError:
    print("Fichero no encontrado")
except PermissionError:
    print("Permisos insuficientes")
except OSError as error:
    print("Error de E/S:", error)
```

Resumen del tema

- La entrada-salida conecta el código Python con datos persistentes: ficheros de texto, CSV, JSON y binarios.
- La regla práctica es escribir código explícito:
 - Rutas claras,
 - Modos de entrada salida adecuados
 - Codificación definida
 - Cierre automático con **with**
 - Errores controlados.
- Con estas bases, en el siguiente tema se podrá organizar código reutilizable en módulos y separar mejor funciones, datos y scripts.

```
# Patrón base del tema
from pathlib import Path

ruta = Path("datos/reporte.txt")
```

```
with ruta.open("r", encoding="utf-8") as f:
    for línea in f:
        print(línea.strip())
```

Flujo de trabajo en el laboratorio

- Desde el repositorio Git podrás cargar el directorio del Tema08.
- Trabajaremos con las operaciones de diferentes formatos de ficheros como Texto, CSV, JSON, Pickle y Binarios.
- La versión de Python usada en el venv del curso será Python 3.13.

```
# Cargar el entorno
cd ~/Curso_Python/Tema08
source .venv/bin/activate
python --version
# Actualizar del repositorio Git
git pull origin main
# Para ejecutar VSCode
code . # usar carpeta Tema08
# Para ejecutar en Terminal
python
```